

Submitted By : Meenakshi Muthukumaran

Organization : InterNova

Domain : Data Analytics

Python, NumPy and Pandas

NumPy Arrays

To practice creating and inspecting NumPy arrays and understanding array dimensions.

Source Code:

```
import numpy as np

#Creating a NumPy array containing at least 10 numbers
lst=[28,89,45,54,67,78,942,546,9876,6336]
arr=np.array(lst)

#Displaying the array
print("NumPy Array:")
print(arr)
#displays the array's shape, size, and data type.
print("Shape:",arr.shape)
print("Size:",arr.size)
print("Data Type:",arr.dtype)

#Creates a one-dimensional and two-dimensional array
arr1=np.array([1,3,5,7,9,11])
arr2=np.array([[1,2,3],
               [4,5,6]])
print("One-Dimensional Array:")
print(arr1)
print("Shape of 1d array: ",arr1.shape)
print("Two-Dimensional Array:")
print(arr2)
print("Shape of 2d array:",arr2.shape)
```

Task 1

Output:

```
[1] import numpy as np
[2] #Creating a NumPy array containing at least 10 numbers
    lst=[28,89,45,54,67,78,942,546,9876,6336]
    arr=np.array(lst)
[3] #Displaying the array
    print("NumPy Array:")
    print(arr)
[4] #displays the array's shape, size, and data type.
    print("Shape:",arr.shape)
    print("Size:",arr.size)
    print("Data Type:",arr.dtype)
[5] #Creates a one-dimensional and two-dimensional array
    arr1=np.array([1,3,5,7,9,11])
    arr2=np.array([[1,2,3],
                   [4,5,6]])
```

```
#Creates a one-dimensional and two-dimensional array
arr1=np.array([1,3,5,7,9,11])
arr2=np.array([[1,2,3],
               [4,5,6]])
print("One-Dimensional Array:")
print(arr1)
print("Shape of 1d array: ",arr1.shape)
print("Two-Dimensional Array:")
print(arr2)
print("Shape of 2d array:",arr2.shape)
```

```
One-Dimensional Array:
[ 1  3  5  7  9 11]
Shape of 1d array: (6,)
Two-Dimensional Array:
[[1 2 3]
 [4 5 6]]
Shape of 2d array: (2, 3)
```

Brief Explanation:

Creates a NumPy array containing ten numbers and displays the array, shape, size and data type. It then creates one-dimensional and two-dimensional arrays and displays their shapes.

Source Code: (Part 1)

```
import numpy as np

# Create a 1D NumPy array
lst=[10,20,30,40,50,60,70,80,90,100]
arr1=np.array(lst)

#Indexing
print(arr1[0])
print(arr1[-1])
print(arr1[5])
print(arr1[-3])

#Slicing
print(arr1[3:9])
print(arr1[::-1])
print(arr1[1:10:2])
print(arr1[:-2])
print(arr1[:])

#2d array
arr2=np.array([
    [1,2,3],
    [6,7,8],
    [9,10,11],
```

Source Code: (Part 2)

```
[2,4,6]
])
print(arr2)

#Row slicing
print(arr2[2:])
print(arr2[1:2])
print(arr2[-2:])

#column slicing
print(arr2[1:3,0:2])

##Reshape the 1D array
reshaped = arr1.reshape(5,2)

print("Original Array: \n",arr1)
print("Reshaped Array: \n",reshaped)

reshaped_2d=arr2.reshape(3,4)
print("Original Array: \n",arr2)
print("Reshaped Array: \n",reshaped)
```

Task 2

Output:

```
[2] ✓ 0s # Create a 1D NumPy array
lst=[10,20,30,40,50,60,70,80,90,100]
arr1=np.array(lst)

[1] #Indexing
print(arr1[0])
print(arr1[-1])
print(arr1[5])
print(arr1[-3])

... 10
100
60
80

[1] #Slicing
print(arr1[3:9])
print(arr1[::-1])
print(arr1[1:10:2])
print(arr1[:-2])
print(arr1[:])

... [40 50 60 70 80 90]
[100 90 80 70 60 50 40 30 20 10]
[ 20 40 60 80 100]
[10 20 30 40 50 60 70 80]
[ 10 20 30 40 50 60 70 80 90 100]
```

```
#2d array
arr2=np.array([
    [1,2,3],
    [6,7,8],
    [9,10,11],
    [2,4,6]
])
print(arr2)

[[ 1  2  3]
 [ 6  7  8]
 [ 9 10 11]
 [ 2  4  6]]

#Row slicing
print(arr2[2:])
print(arr2[1:2])
print(arr2[-2:])

[[ 9 10 11]
 [ 2  4  6]]
[[6 7 8]]
[[ 9 10 11]
 [ 2  4  6]]
```

```
##Reshape the 1D array
reshaped = arr1.reshape(5,2)

print("Original Array: \n",arr1)
print("Reshaped Array: \n",reshaped)

Original Array:
[ 10 20 30 40 50 60 70 80 90 100]
Reshaped Array:
[[ 10 20]
 [ 30 40]
 [ 50 60]
 [ 70 80]
 [ 90 100]]

reshaped_2d=arr2.reshape(3,4)
print("Original Array: \n",arr2)
print("Reshaped Array: \n",reshaped)

Original Array:
[[ 1  2  3]
 [ 6  7  8]
 [ 9 10 11]
 [ 2  4  6]]
Reshaped Array:
[[ 1  2  3  6]
 [ 7  8  9 10]
 [ 2  4  6  9]]
```

Brief Explanation:

Demonstrates positive and negative indexing, one-dimensional slicing, two-dimensional row and column slicing, and reshaping NumPy arrays.

Source Code: (Part 1)

```
import pandas as pd
import numpy as np

from google.colab import files
uploaded=files.upload()

df=pd.read_csv('Simple 100 Student Marks.csv')
df.head()

marks=df['maths_marks'].to_numpy()
print(marks)

#Mathematical Operations
print("Addition :",marks+5)
print("Subtraction: ",marks-5)
print("Multiplication:",marks*5)
print("Division: ",marks/5)

#Statistical Operations
print("Mean :",np.mean(marks))
print("Median :",np.median(marks))
print("Minimum :",np.min(marks))
print("Maximum :",np.max(marks))
print("Standard Deviation:", np.std(marks))
print("Sum: ",np.sum(marks))
```


Task 3

Output:

```
[1]
import pandas as pd
import numpy as np

[2]
from google.colab import files
uploaded=files.upload()

... Choose files Simple 100 ...nt Marks.csv
Simple 100 Student Marks.csv(text/csv) - 3148 bytes, last modified: 14/08/2026 - 100% done
Saving Simple 100 Student Marks.csv to Simple 100 Student Marks.csv

[3]
df=pd.read_csv('Simple 100 Student Marks.csv')
df.head()
```

	student_name	maths_marks	science_marks	english_marks	social_studies_marks	language_marks
0	Mt. Gregory Homerick	69	24	52	4	83
1	Marlene Lettier IV	62	59	70	54	52
2	Josephine Ebert	29	76	13	8	19
3	Dr. Irene Green	44	66	14	2	14
4	Jake Huel	49	23	63	86	68

```
#Mathematical Operations
print("Addition :",marks+5)
print("Subtraction: ",marks-5)
print("Multiplication:",marks*5)
print("Division: ",marks/5)
```

```
Addition : [ 74  67  34  49  54  74  96  70  33  16  64  93   7  35  96  68  43  17
 10  27  12  65  21 104  15  93  71  46  13  72  29  88  23  14  69  12
 12  60 102  99 104  46  77  37  18 105  19  83  83  15  76  39  57  67
 41  18  14  63   7  33  70   9  56  84  66  75  59 102  67  53  23  36
 52  43  75  58  80  41  60  15  58  14  34   7  48  33  99  90  85  72
 93  86  86  54  78  74  37  82  59  33]

Subtraction: [64 57 24 39 44 64 86 60 23   6 54 83 -3 25 86 58 33   7   0 17   2 55 11 94
 5 83 61 36   3 62 19 78 13   4 59   2   2 50 92 89 94 36 67 27   8 95   9 73
 73   5 66 29 47 57 31   8   4 53 -3 23 60 -1 46 74 56 65 49 92 57 43 13 26
 42 33 65 48 70 31 50   5 48   4 24 -3 38 23 89 80 75 62 83 76 76 44 68 64
 27 72 49 23]

Multiplication: [345 310 145 220 245 345 455 325 140   55 295 440   10 150 455 315 190   60
 25 110   35 300   80 495   50 440 330 205   40 335 120 415   90  45 320   35
 35 275 485 470 495 205 360 160   65 500   70 390 390   50 355 170 260 310
180   65  45 290   10 140 325   20 255 395 305 350 270 485 310 240   90 155
235 190 350 265 375 180 275   50 265   45 145   10 215 140 470 425 400 335
440 405 405 245 365 345 160 385 270 140]

Division: [13.8 12.4   5.8   8.8   9.8 13.8 18.2 13.   5.6   2.2 11.8 17.6   0.4   6.
18.2 12.6   7.6   2.4   1.   4.4   1.4 12.   3.2 19.8   2.  17.6 13.2   8.2
 1.6 13.4   4.8 16.6   3.6   1.8 12.8   1.4   1.4 11.  19.4 18.8 19.8   8.2
14.4   6.4   2.6 20.   2.8 15.6 15.6   2.  14.2   6.8 10.4 12.4   7.2   2.6
 1.8 11.6   0.4   5.6 13.   0.8 10.2 15.8 12.2 14.  10.8 19.4 12.4   9.6]
```

Brief Explanation:

Reads the student marks CSV, converts the maths_marks column into a NumPy array, performs arithmetic operations and calculates common statistical measures.

Task 4

Source Code:

```
import pandas as pd

# Pandas Series.
series=pd.Series([20,40,60,80,100,120,140,160,180,200])
print(series)

# Pandas DataFrame
df=pd.DataFrame({"std_names ": ["Meera","Madhu","Ram","Sheela","Swapna","Rohit"],
                  "rollno":[101,102,104,111,150,132],
                  "Marks":[95,90,78,80,75,30]})
print(df)

# column names
print(df.columns)

#index
print(df.index)

#accessing using index
df.loc[2]

df["Grades"]=['A','A','B','B','B','E']
print("updated dataframe : ")
print(df)
```

Task 4

Output:

```
series=pd.Series([20,40,60,80,100,120,140,160,180,200])
print(series)
```

```
0    20
1    40
2    60
3    80
4   100
5   120
6   140
7   160
8   180
9   200
dtype: int64
```

```
# Pandas DataFrame
df=pd.DataFrame({"std_names": ["Meera","Madhu","Ram","Sheela","Swapna","Rohit"],
                 "rollno": [101,102,104,111,150,132],
                 "Marks": [95,90,78,80,75,30]})

print(df)
```

	std_names	rollno	Marks
0	Meera	101	95
1	Madhu	102	90
2	Ram	104	78
3	Sheela	111	80
4	Swapna	150	75
5	Rohit	132	30

```
#accessing using index
df.loc[2]
```

	2
std_names	Ram
rollno	104
Marks	78

dtype: object

```
df["Grades"]=['A','A','B','B','B','E']
print("updated dataframe : ")
print(df)
```

```
updated dataframe :
   std_names  rollno  Marks Grades
0     Meera    101     95      A
1     Madhu    102     90      A
2        Ram    104     78      B
3    Sheela    111     80      B
4    Swapna    150     75      B
5     Rohit    132     30      E
```

Brief Explanation:

Creates a Pandas Series and DataFrame, displays columns and index information, accesses a row using its index, and adds a Grades column.

Source Code: (Part 1)

```
import pandas as pd

from google.colab import files
uploaded=files.upload()

#Reading csv file
df=pd.read_csv('Superstore.csv',encoding='latin1')

#first 5 rows
df.head()

#last 5 rows
df.tail()

#Number of rows and columns
df.shape

#Column Names
df.columns
#Data types
df.dtypes
df.info()
df.describe(include='all')
```

Task 5

Output:

```
from google.colab import files
uploaded=files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to upload files.
Saving Sample - Superstore.xlsx to Sample - Superstore.xlsx

```
#Reading csv file
df=pd.read_csv('Superstore.csv',encoding='latin1')
#first 5 rows
df.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	Postal Code	Region	Product ID	Product Name
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gite	Consumer	United States	Henderson	42420	South	FUR-B100017	Furniture
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gite	Consumer	United States	Henderson	42420	South	FUR-C100004	Furniture
2	3	CA-2016-152156	11/8/2016	11/11/2016	Second Class	DV-12520	Darin	Consumer	United States	Los Angeles	90001	West	OFF-L100001	Office Supplies

```
df.columns
Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category', 'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'],
      dtype='object')
```

```
df.dtypes
Row ID      int64
Order ID    object
Order Date  object
Ship Date   object
Ship Mode   object
Customer ID object
Customer Name object
```

Brief Explanation:

Creates a Pandas Series and DataFrame, displays columns and index information, accesses a row using its index, and adds a Grades column.

Source Code: (Part 1)

```
import pandas as pd

from google.colab import files
uploaded=files.upload()

df=pd.read_csv('Superstore.csv',encoding='latin1')
df.head()

df.info()

#selecting specific column
selected_columns=df[['Segment','Country','City','Postal Code','Region']]
selected_columns.head(15)

#selecting specific row
specific_rows=df.iloc[0:5]
specific_rows

df.iloc[[0, 2, 4]]

#filtering records using one condition
profitable_sales=df[df['Sales']>200]
profitable_sales.head()
```

Source Code: (Part 2)

```
high_sales_profit=df[(df['Sales']>500) & (df['Profit']>200)]  
high_sales_profit.head()
```

```
#sort data by sales in ascending order  
df.sort_values("Sales")
```

```
#Sort data by sales in descending order  
df.sort_values("Sales",ascending=False)
```

Task 6

Output:

```
#SELECTING SPECIFIC COLUMNS
```

```
selected_columns=df[['Segment','Country','City','Postal Code','Region']]
selected_columns.head(15)
```

	Segment	Country	City	Postal Code	Region
0	Consumer	United States	Henderson	42420	South
1	Consumer	United States	Henderson	42420	South
2	Corporate	United States	Los Angeles	90036	West
3	Consumer	United States	Fort Lauderdale	33311	South
4	Consumer	United States	Fort Lauderdale	33311	South
5	Consumer	United States	Los Angeles	90032	West
6	Consumer	United States	Los Angeles	90032	West
7	Consumer	United States	Los Angeles	90032	West
8	Consumer	United States	Los Angeles	90032	West
9	Consumer	United States	Los Angeles	90032	West
10	Consumer	United States	Los Angeles	90032	West
11	Consumer	United States	Los Angeles	90032	West
12	Consumer	United States	Concord	28027	South
13	Consumer	United States	Concord	28027	South

```
#Sort data by sales in descending order
df.sort_values("Sales",ascending=False)
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode
2697	2698	CA-2014-145317	3/18/2014	3/23/2014	Standard Class
6826	6827	CA-2016-118689	10/2/2016	10/9/2016	Standard Class
8153	8154	CA-2017-140151	3/23/2017	3/25/2017	First Class

```
#filtering records using one condition
profitable_sales=df[df['Sales']>200]
profitable_sales.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
7	8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States
10	11	CA-2014-115812	6/9/2014	6/14/2014	Standard	BH-11710	Brosina Hoffman	Consumer	United States

Brief Explanation:

Selects specific columns and rows, filters records using Sales and Profit conditions, and sorts the Superstore data in ascending and descending order by Sales.

Source Code: (Part 1)

```
import pandas as pd

from google.colab import files
uploaded=files.upload()

#Before handling missing values
df=pd.read_csv('retail_store_sales.csv')
df.head()

df.info()
df.isnull().sum()
df['Item']=df['Item'].fillna(df.groupby('Category')['Item'].transform(lambda
x:x.fillna(x.mode()[0])))

df['Price Per Unit']=df['Price Per Unit'].fillna(df['Price Per Unit'].median())

df['Quantity']=df['Quantity'].fillna(df['Quantity'].median())

df['Total Spent']=df['Total Spent'].fillna(df['Total Spent'].median())
df['Discount Applied']=df['Discount Applied'].fillna(df['Discount Applied'].mode())
df.isnull().sum()
df.info()
#After handling missing values
df.head()
```

Output:

```
df.isnull().sum()
```

	0
Transaction ID	0
Customer ID	0
Category	0
Item	1213
Price Per Unit	609
Quantity	604
Total Spent	604
Payment Method	0
Location	0
Transaction Date	0
Discount Applied	4199

```
dtype: int64
```

```
Transaction Date    0
Discount Applied   4199
dtype: int64
```

```
df['Item']=df['Item'].fillna(df.groupby('Category')['Item'].transform(lambda x:x.fillna(x.mode()[0])))
```

```
df['Price Per Unit']=df['Price Per Unit'].fillna(df['Price Per Unit'].median())
```

```
df['Quantity']=df['Quantity'].fillna(df['Quantity'].median())
```

```
df['Total Spent']=df['Total Spent'].fillna(df['Total Spent'].median())
```

```
df['Discount Applied']=df['Discount Applied'].fillna(df['Discount Applied'].mode())
```

```
df.isnull().sum()
```

```
0
Transaction ID    0
Customer ID       0
Category          0
```

Brief Explanation:

Checks missing values in the retail sales dataset and fills missing entries using category-wise mode and median/mode based approaches, then checks the cleaned data.

Source Code: (Part 1)

```
import pandas as pd
```

```
from google.colab import files  
uploaded=files.upload()
```

```
df1=pd.read_csv('customer_data.csv')  
df2=pd.read_csv('sales_data (1).csv')
```

```
df1.head()  
df2.head()
```

```
merged_csv=pd.merge(df1,df2,on="Customer_ID")  
merged_csv.head()
```

```
#Concat  
sales_df1=df2.iloc[:50]  
sales_df2=df2.iloc[50:]  
sales=pd.concat([sales_df1,sales_df2])
```

```
sales.head()
```

```
#Usage of groupby and aggregation functions  
category_analysis=sales.groupby('Category')['Total_Sales'].agg(["sum","mean","count","min","  
max"])
```

Source Code: (Part 2)

```
print(category_analysis)

merged_csv

pivot_table=pd.pivot_table(merged_csv,
                             columns="Product",
                             index="City",
                             values="Total_Sales",
                             aggfunc="sum")

pivot_table

merged_csv.to_csv("final_processed_data.csv",index=False)

from google.colab import files
files.download('final_processed_data.csv')
```

Task 8

Output:

```
import pandas as pd
```

```
from google.colab import files  
uploaded=files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has a file uploader.
Saving customer_data.csv to customer_data.csv
Saving sales_data (1).csv to sales_data (1) (1).csv

```
df1=pd.read_csv('customer_data.csv')  
df2=pd.read_csv('sales_data (1).csv')
```

```
df1.head()
```

	Customer_ID	Customer_Name	Age	Gender	City	Segment
0	C001	Customer_1	58	Female	Hyderabad	Regular
1	C002	Customer_2	48	Male	Bangalore	Regular
2	C003	Customer_3	34	Female	Coimbatore	Premium
3	C004	Customer_4	27	Male	Hyderabad	Regular
4	C005	Customer_5	40	Male	Chennai	New

Usage of groupby and aggregation functions

```
category_analysis=sales.groupby('category')['total_sales'].agg(['sum','mean','count','min','max'])  
print(category_analysis)
```

	sum	mean	count	min	max
Category					
Accessories	181500	4813.333333	45	500	12500
Electronics	255500	4645.454545	55	500	12500

```
merged_csv
```

	Customer_ID	Customer_Name	Age	Gender	City	Segment	Transaction_ID	Product	Category	Quantity	Price	Payment
0	C001	Customer_1	58	Female	Hyderabad	Regular	T001	Headphones	Electronics	5	750	
1	C002	Customer_2	48	Male	Bangalore	Regular	T002	Laptop	Electronics	5	2500	
2	C003	Customer_3	34	Female	Coimbatore	Premium	T003	Phone	Electronics	3	1500	
3	C004	Customer_4	27	Male	Hyderabad	Regular	T004	Keyboard	Accessories	4	1500	
4	C005	Customer_5	40	Male	Chennai	New	T005	Phone	Accessories	1	2000	
...	...	...	...	...	...	...	...	...	...	...	...	...
95	C006	Customer_60	42	Male	Bangalore	New	T006	Keyboard	Accessories	5	2500	
96	C007	Customer_67	26	Male	Bangalore	New	T007	Headphones	Electronics	3	2500	

```
merged_csv=pd.merge(df1,df2,on="Customer_ID")  
merged_csv.head()
```

	Customer_ID	Customer_Name	Age	Gender	City	Segment	Transaction_ID	Product
0	C001	Customer_1	58	Female	Hyderabad	Regular	T001	Headphones
1	C002	Customer_2	48	Male	Bangalore	Regular	T002	Laptop
2	C003	Customer_3	34	Female	Coimbatore	Premium	T003	Phone
3	C004	Customer_4	27	Male	Hyderabad	Regular	T004	Keyboard
4	C005	Customer_5	40	Male	Chennai	New	T005	Phone

```
#Concat
```

```
sales_df1=df2.iloc[:50]
```

```
sales_df2=df2.iloc[50:]
```

```
sales=pd.concat([sales_df1,sales_df2])
```

Brief Explanation:

Merges customer and sales data using Customer_ID, concatenates two sales portions, performs category aggregation, creates a pivot table, and saves the processed dataset.

Source Code:

```
# -*- coding: utf-8 -*-  
# Operations are included in the Task 8–9 Pandas script.  
  
category_analysis=sales.groupby('Category')['Total_Sales'].agg(["sum","mean","count","min","  
max"])  
print(category_analysis)  
  
pivot_table=pd.pivot_table(merged_csv,  
                             columns="Product",  
                             index="City",  
                             values="Total_Sales",  
                             aggfunc="sum")  
pivot_table  
  
merged_csv.to_csv("final_processed_data.csv",index=False)
```

Output:

```

pivot_table=pd.pivot_table(merged_csv,
                             columns="Product",
                             index="City",
                             values="Total_Sales",
                             aggfunc="sum")

```

`pivot_table`

Product	Headphones	Keyboard	Laptop	Phone	Tablet
City					
Bangalore	12500	33750	23000	19000	11500
Chennai	12000	26000	10750	10750	48250
Coimbatore	12250	11000	29500	23000	16500
Delhi	2000	3000	8750	4750	15750
Hyderabad	9750	32250	20000	28500	12500

```
merged_csv.to_csv("final_processed_data.csv",index=False)
```

```

from google.colab import files
files.download('final_processed_data.csv')

```

Brief Explanation:

Merges customer and sales data using Customer_ID, concatenates two sales portions, performs category aggregation, creates a pivot table, and saves the processed dataset.

Source Code: (Part 1)

```
import pandas as pd

from google.colab import files
uploaded=files.upload()

df.head()

df=pd.read_csv('retail_store_sales.csv')
df.head()

df.info()
df.describe()
df.isnull().sum()

df['Item']=df.groupby('Category')['Item'].transform(lambda x:x.fillna(x.mode()[0]))
df['Price Per Unit']=df['Price Per Unit'].fillna(df['Price Per Unit'].median())

df['Quantity']=df['Quantity'].fillna(df['Quantity'].fillna(df['Quantity'].median()))

df['Total Spent']=df['Total Spent'].fillna(df['Total Spent'].median())

df['Discount Applied']=df['Discount Applied'].fillna(df['Discount Applied'].mode()[0])
df.info()
```


Source Code: (Part 2)

```
df.isnull().sum()
```

```
high_sales = df[df['Total Spent'] > 300]  
high_sales.head()
```

```
df.sort_values('Total Spent', ascending=False).head(10)
```

```
category_sales = df.groupby('Category')['Total Spent'].sum()  
category_sales
```

```
df.head(3)
```

```
pivot_table = pd.pivot_table(  
    df,  
    values='Total Spent',  
    index='Payment Method',  
    columns='Category',  
    aggfunc='sum'  
)
```

```
pivot_table
```

```
# Key Insights
```

```
# 1) Cash generated the highest sales for Butchers, with total sales of 75,350.5.
```

```
# 2) Digital Wallet generated the highest sales for Electric household essentials, with  
78,869.0.
```

Source Code: (Part 3)

3) Credit Card generated the highest sales for Butchers, with 75,545.0.

4) Milk Products had the lowest sales across all payment methods.

5) The highest Pivot Table value is 78,869.0.

```
final_dataset=df.to_csv("final_dataset.csv",index=False)
```

```
from google.colab import files
```

```
files.download("final_dataset.csv")
```

Output:

```
pivot_table = pd.pivot_table(
    df,
    values='Total Spent',
    index='Payment Method',
    columns='Category',
    aggfunc='sum'
)

pivot_table
```

Category	Beverages	Butchers	Computers and electric accessories	Electric household essential
Payment Method				
Cash	73070.5	75350.5	70884.0	70288.0
Credit Card	63697.5	75545.0	66494.5	62792.0
Digital Wallet	67983.0	65034.5	62102.5	78869.0

Key Insights 1)Cash generated the highest sales for Butchers, with total sales of 75,350.5.
 2)Digital Wallet generated the highest sales for Electric household essentials, with 78,869.0.
 3)Credit Card generated the highest sales for Butchers, with 75,545.0 — slightly higher than Cash.
 4)Milk Products had the lowest sales across all payment methods, with values ranging from 61,469.5 to 63,209.0.
 5)The highest individual value in the entire Pivot Table is 78,869.0, for Electric household essentials using Digital Wallet.

Key Insights 1)Cash generated the highest sales for Butchers, with total sales of 75,350.5.
 2)Digital Wallet generated the highest sales for Electric household essentials, with 78,869.0.
 3)Credit Card generated the highest sales for Butchers, with 75,545.0 — slightly higher than Cash.
 4)Milk Products had the lowest sales across all payment methods, with values ranging from 61,469.5 to 63,209.0.
 5)The highest individual value in the entire Pivot Table is 78,869.0, for Electric household essentials using Digital Wallet.

```
final_dataset=df.to_csv("final_dataset.csv",index=False)
```

```
from google.colab import files
```

```
files.download("final_dataset.csv")
```

Brief Explanation:

Merges customer and sales data using Customer_ID, concatenates two sales portions, performs category aggregation, creates a pivot table, and saves the processed dataset.

Datasets Used:

- Simple 100 Student Marks.csv — Task 3
- Superstore.csv — Tasks 5 and 6
- retail_store_sales.csv — Tasks 7 and 10
- customer_data.csv and sales_data (1).csv — Tasks 8 and 9
- final_processed_data.csv and final_dataset.csv — generated outputs

GitHub Repository:

https://github.com/MeenakshiEnhancer/InternNova_Tasks/tree/main/Week-2

Google Drive Link:

<https://drive.google.com/drive/folders/123amVY4t7ilLDjvlgHuS1NCukJg6eNOV?usp=sharing>